

리소스 수명 시스템 재구축 — 문제해결 로그 (포트폴리오 재료)

작업일: 2026-07-18 · 대상: [ResourceManager](#) / [ResourceScope](#) / [ResourceRegistry](#) / 씬 전환 성격: "측정→진단→구조→검증"
루프의 기록. 결과물(스코프 시스템)보다 의사결정 근거와 트레이드오프가 포트폴리오의 핵심. 상위 계획: [Refactor_Plan.md](#) ·
도식: [NewSystem_Diagram.pdf](#) · [SceneTransition_Design.pdf](#)

한 줄 요약

기존 2버킷(Global/Scene) + 호출부 bool 리소스 관리를 (key,type) refCount 레지스트리 + 수명 스코프 구조로 재구축하고, 씬 전환을 그 위에 재설계했다. 빅뱅 없이 어댑터로 130개 호출부를 무수정 유지한 채 내부만 교체.

Part 1. 왜 — 2버킷의 구조적 결함 (문제의 근본)

상황

리소스의 실제 수명은 팝업 동안 / 던전 동안 / 파티 유지 동안 / 씬 동안 / 영구 — 5종 이상인데, 표현 수단은 Global / Scene 2개뿐이었다.

진단 (핵심 인과)

수명이 애매한 리소스(예: 팝업 UI)는 개발자가 매번 `isGlobal` 을 손으로 판단해야 했고, "씬 버킷에 넣으면 씬 전환 때 죽어서 불안 + 수동 Release는 누수 위험" 사이에서 전부 안전한 쪽 = Global로 도피했다. 그 결과 Global만 누적되어 영구 상주 메모리가 쌓였다.

결정적 프레임: 몬스터 HP바 누수는 개인의 주의력 실수가 아니라 구조가 만든 선택이었다. 2버킷 구조 자체가 개발자를 global 도피로 몰았다.

그래서 목표

수명 계층을 실제 사용 패턴대로 다층화하고, 리소스를 제 수명에 재배치해 상주 메모리를 회수한다. "나누기(스코프)"만 하면 초기 해제 사고(핑크 텍스처)가 난다. "겹침 허용(refCount)"과 한 세트여야 안전.

Part 2. 설계 — 레지스트리 + 스코프 3원칙

`ResourceManager` (파사드) → `ResourceScope` (수명 티켓) → `ResourceRegistry` (핸들 소유) → `Addressables`

- 핸들 소유는 레지스트리 한 곳. 스코프는 참조 티켓(`HashSet<ResourceKey>`)만 든다 → "누가 Release하나" 문제 자체가 소멸.
- 스코프 내 중복 로드 = refCount 1. `HashSet.Add` 반환값으로 처리 → 호출부가 로드 횟수를 안 세도 됨.
- 취소 ≠ 해제. 취소 토큰은 '기다림 중단'만, 핸들 수명은 스코프가 소유 → `await` 후 use-after-release가 구조적으로 불가능.

핵심 방어 질문: "Addressables가 이미 refCount인데 왜 그 위에 또?"

→ Addressables refCount는 핸들 단위 수동 짝맞추기 — 호출부가 Release를 '기억'해야 하고, 그 기억의 실패가 HP바 누수였다. 스코프는 기억을 구조(수명 단위 자동 정산)로 대체한 것.

Part 3. 설계 단계의 고민들 (트레이드오프 기록)

3-1. refCount를 누가 올리는가 — 이중 카운트 함정

고민: 레지스트리가 요청받을 때마다 무조건 `refCount++` 하면, 같은 스코프가 같은 키를 두 번 로드했을 때 `refCount=2`가 되는데 Dispose 땐 HashSet 기준 1번만 Release → `refCount`가 영영 0에 도달 못 함 = 누수.

결정: 증가 여부 판단 주체를 스코프로. `_acquired.Add(rk)` 가 `true` (이 스코프 첫 참조)일 때만 레지스트리가 `refCount++`.

```
// ResourceScope
bool isFirst = _acquired.Add(rk); // 처음이면 true
return _registry.LoadAsync<T>(rk, incrementRef: isFirst, tok);
```

→ `refCount`의 의미 = "이 리소스를 소유한 서로 다른 스코프 수". 마지막 스코프가 놓을 때만 실제 해제.

3-2. (key, type) 복합 키 — 타입 충돌 근절

상황: 기존 캐시 키가 string뿐이라 같은 키를 다른 T로 로드하면 핸들을 빼서 재로드하는 증상치료 로직(P5)이 있었다.

결정: 키를 (string Key, System.Type Type) 복합 struct로. `IEquatable + GetHashCode((Key,Type))`. 같은 문자열이라도 `GameObject` / `Sprite` 는 서로 다른 엔트리 → 충돌 근본 소멸.

3-3. 프리로드 타입 딜레마 — 제네릭 T vs Location ★

가장 깊었던 설계 문제.

상황: 프리로드는 라벨에서 출발한다. 라벨 하나 → 프리팹·SO·오디오 등 타입이 제각각인 N개로 펼쳐진다. 이걸 `LoadAsync<T>` 로 하려면 각 타입을 컴파일 타임에 알아야 하는데, 라벨 내용은 런타임에야 안다.

막다른 길: `LoadAsync<location.ResourceType>` 는 불가 — 제네릭 `<T>` 는 컴파일 타임 '타입 이름'만 받고, `ResourceType` 은 런타임 '값'이다. 리플렉션 (`MakeGenericMethod`)은 느리고·할당·async 깨짐 → 기각.

결정: Location 기반 비제네릭 로드 경로를 추가.

```
// 라벨 → location 목록 (카탈로그 조회, 실제 로드 아님)
var handle = Addressables.LoadResourceLocationsAsync(labels, MergeMode.Union);
// location마다: 실제 타입으로 키잉해서 로드
var rk = new ResourceKey(loc.PrimaryKey, loc.ResourceType); // ★ 런타임 타입
Addressables.LoadAssetAsync<Object>(location); // Object로 담아도 실제 객체 타입은 유지
```

왜 `<Object>` 로드가 안전한가: 실제 Result는 여전히 `GameObject`. 타입 구분은 '키'가 하고 로드는 부모 타입으로 담아도 무방. 나중에 `LoadAsync<GameObject>(key) = (key, typeof(GameObject))` 가 프리로드한 `(key, GameObject)` 와 정확히 히트 → 재로드 없음.

통찰: `LoadAsync<T>(key)` 도 내부적으로 key→location 변환 후 로드한다. Location 방식은 "조회 단계를 이미 끝낸 지름길" + "런타임에 발견한 섞인 타입 묶음을 다루는 유일한 길".

3-4. 스프라이트 함정 — 측정으로 확인

상황: Sprite 모드 텍스처는 한 주소에 Texture2D(메인) + Sprite(서브) 두 얼굴. `LoadAssetAsync<Object>` 는 메인인 Texture2D를 반환 → UI가 Sprite를 기대하면 어긋남 (과거에 겪은 문제).

판단: Location 방식이 이걸 자동 해결한다고 보장 못 함(location의 ResourceType이 Sprite나 Texture2D냐가 세팅·버전 의존). 크래시는 아니고 "프리로드 무의미 + 이중 로드" 형태의 낭비로 나타남.

결정: 스프라이트/아틀라스는 프리로드 대상에서 제외, 쓰는 쪽에서 타입 명시(`LoadAsync<Sprite>`)로 on-demand. 아틀라스는 `GetSpriteFromAtlasAsync` 전용 경로 유지. → 검증은 프리로드 루프에 `loc.ResourceType.Name` 찍어 실제 카탈로그가 뭘 주는지 눈으로 확인("도구 숫자도 교차검증" 원칙).

Part 4. 씬 전환 재설계

4-1. SceneDataSO 가 사실상 "라벨 배달부"임을 진단

전체 코드에서 `SceneDataSO` 의 4필드 사용을 추적:

필드	실사용	판정
preloadLabels	LoadingScene 프리로드	유일하게 살아있음
sceneAddress	미사용("일단 나중에")	죽은 필드
clearPreviousMemory	미사용(항상 clear)	죽은 필드
sceneType	테이블 키와 중복	죽은 필드

→ 2단 SO 구조가 하는 일은 "씬 enum → preload 라벨" 매핑 하나뿐. `SceneTableSO.SceneEntry` 에 라벨을 인라인하고 `SceneDataSO` 제거.

4-2. 전용 로딩 씬 유지 — 실무 패턴 비교 후 근거 있는 선택

	전용 로딩 씬 (채택)	퍼시스턴트+애디티브 (모던 기본)
옛 씬 정리	자동 (Single이 전부 언로드)	수동 언로드
전환 중 메모리 피크	가장 낮음 (두 씬 비공존)	더 높음 (두 씬 잠시 공존)
전환 느낌	하드컷	심리스 가능

결정 근거: 이 프로젝트의 #1 주제가 "리소스 수명·메모리". Single 모드 완전 언로드 → 전환 피크 억제가 주제와 정합. 심리스 전환은 이 게임(메뉴→선택→게임→던전)에 불필요 → 전용 로딩 씬의 단점이 발현 안 됨. 취향이 아니라 측정 근거 있는 트레이드오프.

(던전 왕복은 GameScene 상태 리셋 방식 채택 = YAGNI. 상태 보존이 필요하면 "허브 애디티브"라는 다른 설계.)

4-3. 오케스트레이션 이전 + 해제 일원화

- 전환 로직을 `SceneManagerEx` (영속 POCO)로 이전. 씬 경계를 넘는 흐름을 매 전환마다 파괴되는 씬 `MonoBehaviour`에 두지 않음. `LoadingScene` 은 트리거+UI 전달만.
- 해제를 한 곳으로: 이전엔 (1) `SceneManagerEx` 가 이전 씬 렌더 중 `Resource.Clear()` (언로드 창) + (2) `LoadingScene` 에서 또 `Clear` = 이중·이른 해제. → `ChangeSceneScope()` 단 한 번(이전 씬 언로드 완료 후, 새 스코프 생성→이전 Dispose 순서 강제).
- 전환 상태를 산재 프로퍼티(`NextSceneData` 등) → `Pending` 값 struct 하나로.

Part 5. 마이그레이션 중 디버깅 서사 (실전 3버그)

5-1. `KeyNotFoundException: 'Global'` — 초기화 순서

증상: 부팅 즉시 `_scopes['Global']` 없음.

원인: `Managers.Init()` 이 다른 매니저는 `Init()` 을 부르는데 `_resource.Init()` 만 누락. 예전 `ResourceManager.Init()` 은 `_sceneHandles.Clear()` 뿐이라 안 불러도 무해했지만, 지금은 스코프를 등록하는 유일한 지점이라 빈 딕셔너리 → 예외.

해결: `Managers.Init()` 에 `_resource.Init()` 한 줄 추가.

교훈: 무해하던 초기화가 리팩터로 필수 지점이 됐다 — "안 불러도 되던 것"의 계약 변화를 놓치기 쉽다.

5-2. `NullReferenceException` in `SelectScene` — split-brain ★

증상: `StartScene`→`SelectScene` 진입 시 `_uiCanvas` null → NRE.

원인: 마이그레이션 과도기의 split-brain. `CreateMainUI` 가

- `LoadAsync<GameObject>("SelectSceneCanvas_New")` → 새 레지스트리에 저장
 - `ShowSceneUI` → `Instantiate(string key)` → 옛 딕셔너리를 뒤짐 (비어 있음)
- 못 찾음 → null → NRE. "로드는 새 시스템, 스폰은 옛 시스템"으로 갈라진 것.

해결: (1) `Instantiate(string key)` 를 `_registry.TryGetAsset<GameObject>(key)` 조회로 교체 + (2) `StartScene`의 Global 프리로드도 옛 `LoadDependenciesAsync` → 새 `LoadAsyncPreload` 로. 두 개가 한 쌍이라 같이 고쳐야 통합됨.

교훈: 파사드 안쪽을 교체할 때 읽기 경로(`Instantiate`)와 쓰기 경로(`LoadAsync`)가 같은 저장소를 보느냐가 생명. 한쪽만 옮기면 조용히 null이 샌다.

5-3. `sceneName` 빈 문자열 폴백

증상 가능성: 테이블에 `sceneName` 미기입 시 `LoadSceneAsync("")` 실패.

원인: 폴백을 `string.Empty` 로(무의미한 삼항). 씬은 enum 이름으로 주소화됨.

해결: `string.IsNullOrEmpty(entry.sceneName) ? type.ToString() : entry.sceneName` .

Part 6. 빅뱅 금지 전략

- 파사드 `LoadAsync<T>(key, isGlobal)` 시그니처를 어댑터로 유지 → 130개 호출부 무수정. 내부만 `isGlobal ? GlobalScope : SceneScope` 로 라우팅.
- 각 단계 = 독립 커밋 = 롤백 지점.
- 과도기엔 옛 2버킷과 새 레지스트리가 공존(split-brain) → 5-2처럼 읽기/쓰기 경로를 단계적으로 새 시스템으로 수렴.

Part 7. 남은 작업 (다음 세션 재개용)

리소스 시스템

#	항목	상태	비고
R1	옛 딕셔너리 은퇴		<code>_globalHandles / _sceneHandles + LoadAsyncInternal</code> (죽음). 옛 <code>LoadDependenciesAsync</code> (죽음). <code>Clear()</code> (무효) 제거
R2	디버그 창/계측을 레지스트리로	측정 차단 중	<code>GetHandleSnapshot / LogAliveReport / AssertSceneHandlesCleared</code> / 카운트가 아직 옛 딕셔너리를 봄 → 현재 빈 값. Phase 2 검증("잔존 핸들=Baseline")이 이거 없으면 불가
R3	<code>_atlasSpriteCache</code> 수명화		해제 경로 없는 영구 캐시 → 아틀라스 핸들 수명 동행 (Phase 3d). 아틀라스 대부분 해체돼 우선순위 낮음
R4	NoCache/JSON use-after-release		<code>LoadAsyncNoCache</code> (P2) → <code>LoadTextAsync</code> (임시 스코프+ .text 복사)
R5	풀↔에셋 수명 통합		<code>PoolManager</code> 를 스코프 소속으로 (Phase 4)

씬 전환

#	항목	상태
S1	전환 계측 재배선	<code>RunLoadSequenceAsync</code> 에 <code>Begin/EndSceneTransition</code> 소실됨 (옛 경로에 있었음) → 새 흐름에 재삽입
S2	<code>RunLoadSequenceAsync</code> <code>try/catch(OCE)/finally</code>	다음 씬 로드 시 Loading 씬 파괴로 토큰 취소 → 미처리 예외 로그 방지 + finally에 <code>EndSceneTransition</code>
S3	죽은 코드 제거	<code>LoadSceneAsync</code> · <code>GetSceneName</code> · <code>NextSceneData/Name</code> (<code>SceneManagerEx</code>), <code>LoadProcessAsync</code> (<code>LoadingScene</code>), 잡동사니 <code>using(Org.BouncyCastle.Ocsp / static NPOI...HSSFColor), SceneDataSO.cs</code>
S4	<code>ShowCover</code> / <code>_transitionUI</code>	현재 <code>_transitionUI</code> 미할당 → no-op. 페이드 쓸지/제거할지 결정

검증 (막힘 순서)

R2(디버그 창 레지스트리 이관)가 선행되어야 Phase 2 성공 기준("씬 왕복 후 잔존 핸들 = Baseline v2, 수치 불변")을 쥔 수 있다. 그 다음 R1(죽은코드 제거)로 통합 완결, 이후 Phase 3(수명 재배치)에서 실제 수치가 움직인다.

Part 8. 포트폴리오 각도 (어필 포인트)

- 방법론이 결과물보다 세다: "측정→진단→구조→검증" 루프. 각 결정에 트레이드오프 근거(Part 3~4).
- 말할 수 있는 숫자(Phase 3 이후): 팝업 세션 영구증가 +46.8MB→0, 진입 Global 512.8MB 다이어트, 던전 왕복 +18.7MB→0. (Baseline v2 대비)
- 정직함: split-brain 버그(5-2)를 숨기지 않고 "과도기 구조가 만든 버그를 계측/디버깅으로 잡았다"로 서술. 프레임 예산 스트리밍·프리로드 배치화는 측정 근거로 제외(필요한 것만 했다는 증거).
- 방어 질문 준비: "Addressables refCount 위에 왜?"(Part 2), "본인 문제를 본인이?"(2버킷은 초기 설계, 계측으로 한계 발견→구조로 재발 차단=성장 서사), "1인 프로젝트 오버엔지니어링?"(측정치가 필요성 증명).